Module 2 and Module 4 in the Pathfinding Visualizer

This project is a C++/SFML pathfinding visualizer. You draw walls on a grid, and BFS, DFS, Dijkstra or A* finds a path from start to end. This note explains where topics from **Module 2** and **Module 4** of the KTU DSA syllabus (PCCST303) are used.

Module	Syllabus topic	Feature in the project	Key
2: Linked List & Memory Management	Doubly linked list, dynamic memory allocation and freeing	Undo / redo of grid edits	`Z` / `Y`
4: Sorting & Searching	Merge sort, hashing (division method, linear probing)	Algorithm benchmark and ranking, with cached results	`B`

Everything below is written by hand. It does not use `std::list`, `std::sort` or `std::unordered_map`.

Module 2: Doubly linked list for undo / redo

Files: `src/EditHistory.h`, `src/EditHistory.cpp`

The idea

Undo/redo works like a browser's **Back / Forward** buttons. Each grid state the user creates is stored in a node of a **doubly linked list**:

```
 head                                              tail
  |                                                  |
  ▼                                                  ▼
[empty] ⇄ [wall drawn] ⇄ [maze generated] ⇄ [start moved]
                                ▲
                           m_current   ← the state shown on screen
```

- **Undo (`Z`)** moves `m_current` to `m_current->prev`.
- **Redo (`Y`)** moves `m_current` to `m_current->next`.

We need a **doubly** linked list because undo and redo move in opposite directions, so each node needs both `prev` and `next` pointers.

The node

```
struct Node {
    GridLayout layout;     // walls, weights, start, end
    Node* prev = nullptr;
    Node* next = nullptr;
};
```

`GridLayout` (in `Grid.h`) is a snapshot of the cells the user edited. Search colours like "visited" and "path" are left out.

Operations

Operation	What it does	Linked list concept	Time
`record()` (`EditHistory.cpp:34`)	Adds a new node after the current one	Insert at the end	O(1) plus the snapshot copy
`deleteAfter()` (`EditHistory.cpp:14`)	If you undo and then make a new edit, the old "redo" nodes are deleted	Delete all nodes after a given node	O(k)
`popFront()` (`EditHistory.cpp:26`)	Keeps at most 200 states by removing the oldest node	Delete from the front	O(1)
`undo()` / `redo()` (`EditHistory.cpp:54`, `:60`)	Moves backward or forward one step	Moving through the list using `prev` / `next`	O(1)
Destructor	Frees every node	Memory freeing (manual `delete`)	O(n)

Example

1. Draw a wall → node A is recorded.
2. Generate a maze → node B is recorded. List: `start ⇄ A ⇄ B`, current = B.
3. Press `Z` twice → current = start. The screen goes back to the empty grid.
4. Draw a new wall → nodes A and B are **deleted** and a new node C is added: `start ⇄ C`. This is the same as visiting a new page after going "back" in a browser: the forward history is gone.

Link to the memory management part of Module 2

Nodes are created with `new` and freed with `delete`. If nodes were not freed when the redo branch is cut off or the list is over its limit, the program would leak memory. That shows why freeing memory you no longer use (the job of garbage collection) matters.

Module 4: Sorting and hashing in the benchmark

Files: `src/Benchmark.h`, `src/Benchmark.cpp`

Pressing `B` runs all four algorithms on the current grid, **sorts** them from best to worst, and shows a line like:

```
Ranking:  1. DFS (631 visited, cost 414)  2. A* (749 visited, cost 414)  3. Dijkstra
(766 visited, cost 414)  4. BFS (767 visited, cost 414)
```

Part 1: Merge sort

Code: `mergeSort()` at `Benchmark.cpp:29`, `merge()` at `Benchmark.cpp:12`

The four results are ranked with a hand-written **merge sort**, using these rules in order:

1. Algorithms that **found a path** come first.
2. Then **fewer cells visited** (less work done).
3. Then **cheaper path cost** (weighted cells cost 5, normal cells cost 1).

How merge sort works here:

```
mergeSort([BFS, DFS, Dijkstra, A*])
├── mergeSort([BFS, DFS])        → split → merge → [DFS, BFS]
├── mergeSort([Dijkstra, A*])    → split → merge → [A*, Dijkstra]
└── merge([DFS, BFS], [A*, Dijkstra]) → [DFS, A*, Dijkstra, BFS]
```

- **Divide:** split the list in half again and again until each part has 1 item.
- **Conquer:** merge two sorted halves by repeatedly taking the better front item.
- **Time:** O(n log n) in every case. **Extra space:** O(n) for the `temp` buffer.
- **Stable:** when two items are equal, the left one is taken first, so ties stay in their original order.

Part 2: Hashing with linear probing

Code: `probe()` at `Benchmark.cpp:66`, `fingerprint()` at `Benchmark.cpp:47`, table defined in `Benchmark.h`

Running four searches takes time, so each ranking is **cached** in a hash table. Pressing `B` again on the same grid gives the answer straight from the table, and the line shows `[cached]`.

Step 1: make a key. The whole grid (every cell, plus start and end) is turned into one 64-bit number using the FNV-1a hash. Two different grids are very unlikely to get the same number.

Step 2: hash function (division method). This is the exact method in the syllabus:

```
int home = key % TABLE_SIZE;     // TABLE_SIZE = 61, a prime number
```

A **prime** table size spreads the keys more evenly across the slots.

Step 3: collision resolution (linear probing). If the home slot is already taken by a different grid, check the next slot, then the next, wrapping around the end:

```
idx = (home + step) % TABLE_SIZE;   // step = 0, 1, 2, ...
```

```
slot:   ... | 17 | 18 | 19 | 20 | ...
              ▲    ▲    ▲
            home taken taken → free, so store here
```

Step 4: lookup. Use the same probe sequence. Stop when you find a slot with the same key (a hit) or an empty slot (a miss). Each slot also stores the full grid layout, so even if two grids got the same key by chance, the wrong result is never returned.

When the table is full: all 61 slots are cleared and it starts again, because old mazes are rarely needed.

Operation	Average time	Worst time
Search / insert in hash table	O(1)	O(table size)
Benchmark without cache	4 full graph searches	
Benchmark with cache (hit)	1 hash + 1 probe	

Try it yourself

```
cmake -S . -B build && cmake --build build
./build/pathfinder
```

1. Press `M` to generate a maze.
2. Press `B` to see the merge-sorted ranking. Press `B` again and you will see `[cached]` (the hash table hit).
3. Draw some walls with the left mouse button, then press `Z` to undo and `Y` to redo (the doubly linked list).

Other modules, for completeness

- **Module 1:** a stack (DFS, maze generator), a queue (BFS), and 2D arrays (the grid).
- **Module 3:** graphs (the grid is a graph), DFS / BFS, Dijkstra (single source to all destinations), and a binary heap / priority queue (Dijkstra and A*).